Comparative Analysis and Strategic Ranking of High-Performance Embedding Models for Local Retrieval-Augmented Generation (RAG) Infrastructures via Ollama

1. Executive Summary

The paradigm of Retrieval-Augmented Generation (RAG) has shifted decisively from cloud-dependent architectures toward localized, privacy-centric deployments. This transition is driven by the maturation of open-source Large Language Models (LLMs) and, more critically, the evolution of high-fidelity embedding models capable of running on consumer-grade hardware. For enterprise architects and developers leveraging **Ollama** as their inference backend, the selection of an embedding model represents a foundational architectural decision that dictates the efficacy of the entire semantic search pipeline.

This report provides an exhaustive, expert-level analysis of the current embedding model landscape to identify the optimal candidates for a local RAG system. The evaluation criteria extend beyond standard English benchmarks (MTEB) to encompass **multilingual fidelity** (with a specific focus on Spanish performance, reflecting the complexity of Romance language morphology), **context window utilization** (addressing the shift from 512 to 8192+ token windows), **architectural efficiency** (Matryoshka Representation Learning), and **integration stability** within the Ollama/GGUF ecosystem.

Our research identifies a stratified hierarchy of models, each serving distinct operational requirements. **Nomic-Embed-Text-v1.5** emerges as the premier generalist choice, offering a balanced profile of long-context support, dimensional flexibility, and rock-solid stability in standard Ollama deployments. **BGE-M3** secures the position of the heavy-duty specialist, delivering unrivaled multilingual retrieval capabilities despite higher computational demands and specific implementation nuances on Windows platforms. **Qwen3-Embedding-0.6B** represents the vanguard of efficiency, leveraging a decoder-based architecture to deliver state-of-the-art performance at a fraction of the parameter count, while **GTE-Multilingual-Base** and **Jina-Embeddings-v3** offer compelling alternatives for compact throughput and experimental high-performance scenarios, respectively.

The following analysis dissects the technical specifications, benchmark performance, and deployment realities of these top five candidates. It synthesizes data from academic papers, community bug reports, and performance benchmarks to provide a definitive guide for

implementing robust local search infrastructure in 2026.

---

2. The Physics of Local Semantic Search: Architectural Primitives

To accurately rank embedding models for an Ollama-based RAG system, one must first deconstruct the underlying technological shifts that define the current generation of dense vector retrievers. The operational constraints of a local environment—memory bandwidth, VRAM availability, and quantization precision—create a distinct optimization landscape compared to cloud-based API consumption.

2.1. The Transition from Encoders to Long-Context Architectures

Historically, the embedding landscape was dominated by BERT-based bi-encoder models, such as all-mpnet-base-v2.[1] These models, while efficient, were fundamentally constrained by a maximum sequence length of 512 tokens. In a RAG context, this limitation necessitated aggressive chunking strategies—splitting documents into arbitrary segments that often severed semantic continuity. A legal contract or technical manual section exceeding 512 tokens would be fragmented, potentially separating a clause from its defining condition and degrading retrieval relevance.

The modern cohort of models, including **Nomic-Embed-Text-v1.5** and **GTE-Multilingual-Base**, has transcended this limitation through the integration of **Rotary Positional Embeddings (RoPE)**.[2] This mechanism allows the attention heads to extrapolate positional information far beyond the training context, enabling effective windows of 8,192 tokens or more.

- **Operational Implication:** For an Ollama deployment, this capability shifts the bottleneck from "text segmentation" to "attention computation." Processing an 8k token document requires a quadratic increase in memory for the key-value (KV) cache compared to short contexts. Models that implement RoPE efficiently—or those that utilize sparse attention patterns—become critical for local hardware where VRAM is a scarce commodity shared between the embedder and the generative LLM.

2.2. Matryoshka Representation Learning (MRL): The Storage Revolution

A critical innovation characterizing the 2025–2026 era of embedding models is the widespread adoption of **Matryoshka Representation Learning (MRL)**.[1] Traditional dense embeddings output a fixed-size vector (typically 768, 1024, or 4096 dimensions). Reducing this dimensionality via Principal Component Analysis (PCA) post-hoc often resulted in

catastrophic information loss.

MRL-trained models, such as **Nomic-Embed-Text-v1.5** and **Jina-Embeddings-v3**, optimize the vector space such that the most significant semantic information is concentrated in the earlier dimensions. This "nested doll" structure allows a system architect to generate a full 768-dimensional vector during inference but store only the first 128 or 256 dimensions in the vector database (e.g., Chroma, Qdrant, or pgvector).[5]

- **Systemic Benefit:** This capability is transformative for local RAG. A user can run the embedding inference on a GPU via Ollama at full precision (f16 or f32), but truncate the storage. This reduces the disk I/O and RAM requirements of the vector index by up to 75%, significantly accelerating the Approximate Nearest Neighbor (ANN) search step without requiring a more powerful machine. This decoupling of *inference cost* from *storage cost* is a key differentiator for top-tier models.

2.3. The Multilingual Gap and Cross-Lingual Alignment

The requirement for robust performance in languages like Spanish introduces the challenge of the "language gap." Early English-centric models (e.g., all-MiniLM) fail to capture the semantic nuance of non-English queries, often relying on simple lexical overlap rather than conceptual mapping.

Modern multilingual models employ distinct alignment strategies:

- **Contrastive Learning on Parallel Corpora:** Models like **BGE-M3** are trained on massive datasets of bitext pairs (e.g., English-Spanish sentence equivalents).[6] This forces the model to minimize the cosine distance between "cat" and "gato," effectively aligning the vector spaces of different languages. This is crucial for "Cross-Lingual Information Retrieval" (CLIR), where a user might query in English to find documents written in Spanish.
- **Adapter-Based Specialization: Jina-Embeddings-v3** utilizes Low-Rank Adaptation (LoRA) adapters.[2] Instead of a single monolithic weight set, the model dynamically activates specific low-rank matrices depending on the language or task. This allows for high specificity (e.g., Spanish legal retrieval) without the interference often seen in massively multilingual models (the "curse of multilinguality," where capacity is diluted across too many languages).

2.4. The Ollama/GGUF Quantization Factor

Deploying via Ollama implies a dependency on the **GGUF (GPT-Generated Unified Format)** file standard and the **llama.cpp** inference engine. This environment introduces unique constraints that do not exist in Python/PyTorch deployments:

- **Quantization Sensitivity:** Embedding models are highly sensitive to quantization. While a generative LLM might perform acceptably at 4-bit quantization (q4_k_m), an embedding model—whose sole output is the precise geometric coordinate of a text—can

suffer significant degradation in retrieval quality (nDCG scores) if quantized too aggressively. The ranking in this report prioritizes models that maintain high fidelity even at q8_0 or f16 quantization levels within Ollama.
- **Architectural Support Lag:** Novel architectures often face a "support lag." For instance, while Jina-v3's LoRA architecture is theoretically superior, its complex graph structure requires specific support in the GGUF specification, which may not be present in older Ollama versions.[7] Models with standard architectures (like BERT or RoBERTa) therefore gain a "stability premium" in our ranking.

---

3. Evaluation Methodology

To construct a rigorous ranking of the top 5 candidates, we synthesized data from three primary vectors: Quantitative Benchmark Performance, Architectural Suitability for Local RAG, and Deployment Maturity within Ollama.

3.1. Quantitative Metrics: MTEB and MIRACL

Our analysis relies on two primary benchmarks widely accepted in the information retrieval community:

- **MTEB (Massive Text Embedding Benchmark):** This is the holistic standard for English embedding quality. We specifically isolated the **Retrieval** task scores (nDCG@10), disregarding clustering or classification scores, as RAG systems rely almost exclusively on the retrieval of relevant passages.[9]
- **MIRACL (Multilingual Information Retrieval Across a Continuum of Languages):** This benchmark is critical for assessing non-English performance. We scrutinized results for **Spanish (es)** to address the implicit requirement for multilingual robustness found in the research context. High MIRACL scores indicate a model's ability to handle the morphological richness and syntactic variations of Spanish.[10]

3.2. Qualitative Factors: The "Ollama Fit"

A model's theoretical performance on a leaderboard is irrelevant if it cannot be reliably deployed in a local environment. We evaluated:

- **Parameter Efficiency:** Models exceeding 7 billion parameters (e.g., GritLM-7B or Mistral-Embed) were excluded from the primary ranking. In a local RAG setup, VRAM is a zero-sum game; allocating 14GB of VRAM to an embedder leaves insufficient room for the generative model (e.g., Llama 3 or Mistral). The ideal local embedder should operate under 1GB of VRAM.
- **Stability and Compatibility:** We analyzed community reports regarding NaN (Not a Number) errors, segmentation faults, and platform-specific bugs (e.g., Windows vs. Linux behavior).[12] Models with a track record of "it just works" via ollama pull were given higher

weight.
- **Native Feature Support:** Models that natively support the modelfile parameters for context window adjustment and batch processing in Ollama were favored over those requiring custom compilation or complex workarounds.

3.3. Candidate Exclusion

Several notable models were excluded from the top 5 ranking:

- **OpenAI text-embedding-3 / Cohere:** These are proprietary, cloud-only models. They violate the fundamental constraint of a "local" RAG system.
- **all-MiniLM-L6-v2:** While a legendary baseline, its 512-token limit and lack of multilingual training data render it obsolete for a modern "best-in-class" list for 2026. It serves only as a comparative baseline.[13]
- **mxbai-embed-large:** A strong contender, but it was edged out by models offering either better multilingual support (BGE-M3) or better efficiency (Qwen3).

---

4. Candidate Analysis: The Top 5 Ranking

Based on the convergence of retrieval quality, multilingual capability, and Ollama integration maturity, we rank the candidates as follows:

1. **Nomic-Embed-Text-v1.5** (Best Overall Balance)
2. **BGE-M3** (Best Multilingual & Precision Specialist)
3. **Qwen3-Embedding-0.6B** (Best Efficiency & Modern Architecture)
4. **GTE-Multilingual-Base** (Best Compact Throughput)
5. **Jina-Embeddings-v3** (Best Performance Potential/Experimental)

4.1. Nomic-Embed-Text-v1.5

The "Gold Standard" for Local RAG

- **Model Type:** Dense Encoder (BERT-based with RoPE and Matryoshka)
- **Parameters:** 137M
- **Context Window:** 8,192 tokens (configurable via dynamic NTK-aware interpolation)
- **Ollama Command:** ollama pull nomic-embed-text

Architectural Profile

Nomic-Embed-Text-v1.5 represents a pivotal moment in the democratization of high-quality embeddings. It was the first fully reproducible, open-weights, and open-data model to successfully challenge the dominance of OpenAI's text-embedding-ada-002.[14] Architecturally, it builds upon the NomicBERT framework but incorporates **Rotary Positional Embeddings (RoPE)** to extend the context window to 8,192 tokens. This design choice allows the model to ingest long-form documents—such as entire Spanish legal statutes or technical

manuals—without the semantic fragmentation caused by aggressive chunking.

Crucially, Nomic-v1.5 is trained with **Matryoshka Representation Learning (MRL)**. This allows the model to learn a hierarchy of information density. The first 64 dimensions contain the most coarse-grained semantic information, while subsequent dimensions add increasingly fine-grained detail.

- **Implication for RAG:** A developer can execute the model in Ollama to generate the full 768-dimensional vector but configure their vector database (e.g., ChromaDB) to index only the first 256 dimensions. This provides a 3x reduction in storage and memory usage with a negligible drop in retrieval accuracy (often less than 2% degradation in nDCG).[14]

Performance Analysis

- **Retrieval Quality:** On the MTEB benchmark, Nomic-v1.5 consistently outperforms larger models in the 100M-400M parameter class. While its training data is predominantly English, fine-tuning on diverse corpora has endowed it with respectable multilingual capabilities.
- **Spanish Capability:** In MIRACL evaluations, Nomic-v1.5 demonstrates strong performance for general-purpose Spanish retrieval. While it may not match the specialized nuancing of BGE-M3 in highly technical cross-lingual tasks, it is more than sufficient for standard enterprise RAG applications (e.g., searching internal knowledge bases or customer support logs).
- **Long-Context Fidelity:** Tests on the LoCo (Long Context) benchmark indicate that Nomic-v1.5 maintains retrieval stability even as the input sequence length approaches its 8k limit. This contrasts with older models that suffer from "attention dilution," where performance drops precipitously after the first 1,000 tokens.

Ollama Integration & Usability

Nomic-Embed-Text holds the distinction of being the most stable and widely supported model in the Ollama library.

- **Deployment:** The model is available directly via the official library, ensuring that the GGUF quantization is optimized for the architecture. The ollama pull command fetches a version that is pre-configured with the correct prompt templates (search_query: vs. search_document: prefixes), which are essential for its performance.
- **Stability:** Unlike newer experimental architectures, Nomic-v1.5 runs reliably across all major platforms (Windows, macOS, Linux) without triggering NaN errors or segmentation faults. This reliability makes it the ideal "set and forget" choice for production systems.

Strategic Verdict: Nomic-Embed-Text-v1.5 ranks #1 because it solves the "trilemma" of local embeddings: it offers long context, storage efficiency via MRL, and exceptional stability, all within a lightweight 137M parameter footprint.

4.2. BGE-M3 (BAAI General Embedding)

The Multilingual Precision Specialist

- **Model Type:** Hybrid (Dense + Sparse + Multi-Vector)
- **Parameters:** 567M
- **Context Window:** 8,192 tokens
- **Ollama Command:** ollama pull bge-m3

Architectural Profile

BGE-M3 (Multi-Linguality, Multi-Functionality, Multi-Granularity) is a heavyweight contender designed to address the limitations of purely dense retrievers. Developed by the Beijing Academy of Artificial Intelligence (BAAI), it introduces a novel hybrid architecture capable of generating three distinct types of outputs simultaneously [6]:

1. **Dense Embeddings:** Standard semantic vectors for broad concept matching.
2. **Sparse Embeddings:** Lexical weights (similar to a learnable BM25) that excel at exact keyword matching—a critical feature for technical domains (e.g., matching a specific part number "AX-99" which might be lost in a dense vector).
3. **Multi-Vector (ColBERT):** Late-interaction representations that allow for fine-grained token-level matching.

- *Note:* In a standard Ollama pipeline, users typically consume only the **dense** output. However, the model's internal training on these mixed modalities results in a dense vector that is exceptionally rich in both semantic and lexical information.

Performance Analysis

- **Multilingual Supremacy:** BGE-M3 is the undisputed leader in multilingual benchmarks among models under 1B parameters. It supports over 100 languages and achieves state-of-the-art results on MIRACL.[15] For a RAG system indexing Spanish technical documentation, BGE-M3 captures subtle semantic distinctions that English-centric models miss.
- **Cross-Lingual Retrieval:** It excels in "cross-lingual" scenarios. If a user queries in English ("How to reset the boiler pressure?") against a corpus of Spanish manuals ("Cómo restablecer la presión..."), BGE-M3's aligned vector space ensures the relevant Spanish document is retrieved with high confidence.

Ollama Integration & The NaN Challenge

Despite its performance, BGE-M3 presents specific challenges in the Ollama ecosystem that prevent it from taking the #1 spot:

- **The NaN Issue:** A persistent issue affects BGE-M3 when running on Windows or specific NVIDIA GPU architectures via llama.cpp.[12] Users frequently report the model outputting NaN (Not a Number) values for certain inputs. This is often attributed to numerical instability in the half-precision (f16) computation of the Flash Attention mechanism.

    - *Remediation:* The workaround involves setting the environment variable OLLAMA_FLASH_ATTENTION=false before launching the server. While effective, this requirement for manual intervention lowers its usability score.
- **Resource Intensity:** At 567M parameters, BGE-M3 is approximately 4x larger than Nomic-v1.5. While manageable, it consumes more VRAM and has higher latency per query.

Strategic Verdict: BGE-M3 is the #2 choice, but effectively the #1 choice for **multilingual-heavy** or **technical** workloads. If your primary data is in Spanish, the trade-off of configuring the workaround is well worth the gain in retrieval accuracy.

4.3. Qwen3-Embedding-0.6B

The Vanguard of Efficiency

- **Model Type:** Dense Decoder-based Embedding
- **Parameters:** 0.6B (approx. 600M)
- **Context Window:** 32,000 tokens (Architecture capability), typically 8k in GGUF
- **Ollama Command:** ollama pull qwen3-embedding:0.6b

Architectural Profile

Qwen3-Embedding-0.6B represents a significant architectural divergence. While most embeddings use Encoder-only architectures (like BERT), Qwen3 utilizes a **Decoder-only** architecture derived from the massive Qwen LLM family.[16] This allows it to leverage the immense pre-training knowledge of the Qwen generative models, particularly in code and reasoning tasks.

- **Instruction Tuning:** Qwen3 is natively instruction-tuned. It expects prompts that define the task, such as "Represent this sentence for searching relevant passages." This instruction awareness allows the model to alter the geometry of the embedding space dynamically, optimizing for either symmetric tasks (sentence similarity) or asymmetric tasks (query-document retrieval).

Performance Analysis

- **Efficiency vs. Performance:** Despite its modest size (0.6B), Qwen3 punches significantly above its weight class. Benchmark data from C-MTEB and MIRACL suggests it rivals multi-billion parameter models.[18]
- **Code and Technical Domain:** Due to the Qwen family's strong training on code repositories, Qwen3-Embedding is exceptionally strong at retrieving code snippets or highly structured technical data (JSON, XML schemas), making it a top contender for RAG systems focused on software documentation.
- **Multilingual Competence:** Qwen models are renowned for their bilingual (English/Chinese) capabilities, but their general multilingual performance (including

Spanish) is robust due to the massive scale of the underlying training data.

Ollama Integration

- **Native Support:** The model is available in the Ollama library. The 0.6B size strikes a "Goldilocks" balance—large enough to encode deep semantic relationships, but small enough to have negligible impact on system RAM.
- **Context Window:** While the architecture supports 32k context, standard GGUF quantizations in Ollama may default to shorter windows to save memory. Users can manually override this using a custom Modelfile (PARAMETER num_ctx 32768) to unlock the full potential.[19]

Strategic Verdict: Ranked #3, Qwen3-Embedding is the "modernist" choice. It is ideal for developers who want to leverage the latest architectural advancements (decoder embeddings) and need high performance in code/technical domains without the bloat of larger models.

4.4. GTE-Multilingual-Base

The Compact Throughput Specialist

- **Model Type:** Dense Encoder (XLM-RoBERTa based)
- **Parameters:** 305M
- **Context Window:** 8,192 tokens
- **Ollama Command:** ollama pull alibaba-NLP/gte-multilingual-base

Architectural Profile

Alibaba's **GTE (General Text Embedding)** series has been a consistent leader on the MTEB leaderboard. The "Multilingual-Base" variant is engineered to be a direct, superior replacement for the aging XLM-RoBERTa model. It features a 305M parameter count, positioning it exactly between Nomic (137M) and BGE-M3 (567M).[20]

Performance Analysis

- **Spanish/Multilingual Focus:** GTE-Multilingual is explicitly designed to handle 70+ languages. Evaluations show it outperforms previous state-of-the-art models of similar size on the MIRACL Spanish subset.[21]
- **Throughput:** Its primary advantage is *throughput*. It is significantly faster than BGE-M3 due to having nearly half the parameters, yet it retains a large portion of the multilingual retrieval accuracy. This makes it ideal for high-volume ingestion pipelines where documents must be indexed rapidly.

Ollama Integration

- **Community Dependency:** Unlike Nomic, GTE-Multilingual often relies on community-maintained GGUF uploads in the Ollama ecosystem (e.g., via the alibaba-NLP

or other namespaces).[22] This introduces a slight friction, as users must ensure they are pulling a correctly quantized version that supports the full context window.

Strategic Verdict: Ranked #4, GTE-Multilingual-Base is the "Efficiency Alternative" to BGE-M3. It is the best choice for systems where Spanish support is critical, but hardware constraints prevent the use of the larger BGE model.

4.5. Jina-Embeddings-v3

The Experimental Frontier

- **Model Type:** XLM-RoBERTa with Task-Specific LoRA Adapters
- **Parameters:** 570M
- **Context Window:** 8,192 tokens
- **Ollama Command:** Evolving (Requires custom setup or specific community builds)

Architectural Profile

Jina-Embeddings-v3 represents the bleeding edge of embedding research. It utilizes a base XLM-RoBERTa model augmented with five distinct **LoRA (Low-Rank Adaptation)** adapters.[2] Depending on the prompt or configuration, the model activates specific adapters for different tasks: "retrieval-query," "retrieval-passage," "clustering," "classification," etc. This allows a single model to specialize dynamically, theoretically avoiding the "jack of all trades, master of none" problem.

Performance Analysis

- **Benchmark Dominance:** On paper, Jina-v3 is the highest-performing model in this list. It consistently tops MTEB and MIRACL leaderboards, showing exceptional capability in both English and Spanish long-context retrieval.[23]

Ollama Integration & The "Why it's #5"

- **Integration Friction:** The LoRA architecture is non-standard for the llama.cpp backend that powers Ollama. While support is actively being developed (with recent PRs addressing LoRA tensor mapping [7]), it is not yet a seamless "one-click" experience for many users. Users often encounter errors related to missing tensor mappings or inability to specify the active adapter via the API.
- **Complexity:** Effectively using Jina-v3 requires the user to manage the task parameter correctly. If the wrong adapter is active (e.g., using "classification" for a "retrieval" task), performance degrades significantly.

Strategic Verdict: Ranked #5, Jina-v3 is the "High Risk, High Reward" option. It is recommended only for advanced AI engineers who are comfortable troubleshooting GGUF configurations and want the absolute theoretical maximum performance.

---

5. Comparative Synthesis and Decision Matrix

The following data synthesis facilitates a direct comparison of the candidates across key operational dimensions.

5.1. Technical Feature Matrix

Feature	Nomic-Embed-v1.5	BGE-M3	Qwen3-0.6B	GTE-Multi-Base	Jina-v3
Architecture	BERT Encoder	Hybrid Encoder	Decoder	XLM-R Encoder	LoRA Adapter
Params	137M	567M	~600M	305M	570M
Max Context	8,192	8,192	32,000	8,192	8,192
Matryoshka	**Yes**	No (Multi-Granularity)	**Yes**	**Yes**	**Yes**
Multilingual	Good	**Excellent**	**Excellent**	**Excellent**	**Excellent**
Ollama Stability	**High** (Native)	Medium (NaN fix req.)	High	Medium	Low (Experimental)
Spanish MIRACL	Competent	**State-of-the-Art**	High	High	**State-of-the-Art**

5.2. Scenario-Based Recommendations

Scenario A: The "Set and Forget" Enterprise RAG

- **Requirement:** A stable system for searching mixed English/Spanish company policies on standard office hardware.
- **Recommendation: Nomic-Embed-Text-v1.5**.

- **Rationale:** The stability and low resource footprint outweigh the marginal gains of larger multilingual models. The 8k context ensures full policy documents are embedded correctly.

Scenario B: The Technical Documentation Search (Spanish)

- **Requirement:** Precise retrieval of engineering manuals in Spanish, distinguishing between specific technical terms.
- **Recommendation: BGE-M3**.
- **Rationale:** The model's training on bitext pairs ensures that specific Spanish technical terminology is correctly mapped to the query intent. The NaN workaround is an acceptable cost for the precision gain.

Scenario C: Code & Technical Support Bot

- **Requirement:** Searching a mix of Python code, JSON logs, and English/Spanish documentation.
- **Recommendation: Qwen3-Embedding-0.6B**.
- **Rationale:** Qwen's code-heavy training data makes it uniquely suited for this domain. The instruction tuning allows for distinct handling of "code queries" vs "text queries."

---

6. Implementation Guide: Optimizing for Ollama

To realize the potential of these models, the following technical optimizations should be implemented.

6.1. Addressing the BGE-M3 NaN Issue on Windows

If deploying BGE-M3, the NaN instability is a critical blocker. This is caused by f16 overflow in the attention kernel on specific hardware.[12]

- **Remediation:** Configure the environment variable explicitly before launching Ollama:
  Bash
  ```
  set OLLAMA_FLASH_ATTENTION=false
  ollama serve
  ```

  This forces the backend to use a more stable (though slightly slower) attention computation path, eliminating the crash.

6.2. Implementing Matryoshka Truncation

For Nomic, Qwen3, and Jina, the embedding vector can be truncated *after* generation but *before* indexing.

- **Procedure:**

1. Generate the full embedding via Ollama (e.g., 768 dim).
2. In your application code (Python/JS), slice the array to the first 256 elements: embedding = response['embedding'][:256].
3. Store this truncated vector in your vector DB.

- **Benefit:** This reduces the RAM required for the HNSW index by ~66%, allowing you to store millions of documents on a local machine with only 16GB or 32GB of RAM.

6.3. Custom Modelfiles for Context

Ollama may default to a 2048 or 4096 context window to conserve VRAM. To utilize the full 8192 capability of Nomic or BGE-M3, create a custom Modelfile:

Dockerfile

```
FROM nomic-embed-text
PARAMETER num_ctx 8192
```

Then create and run this custom version:

Bash

```
ollama create nomic-8k -f Modelfile
ollama run nomic-8k
```

7. Future Outlook

The trajectory of local embeddings points toward **Multimodal Embeddings** (e.g., ColPali), which embed document images directly rather than OCR'd text. While not yet standard in Ollama, models like **Qwen3** are paving the way with their multimodal training foundations. For the 2026 horizon, establishing a robust pipeline with **Nomic** or **BGE-M3** today provides a stable foundation that can be upgraded to these multimodal architectures as GGUF support matures.

In conclusion, while **Nomic-Embed-Text-v1.5** remains the pragmatic champion for its balance of performance and ease of use, **BGE-M3** stands as the definitive choice for any application where multilingual Spanish fidelity is non-negotiable. The choice between them is

a choice between *operational simplicity* and *specialized precision*.

Obras citadas

1. The Best Open-Source Embedding Models in 2026 - BentoML, fecha de acceso: febrero 9, 2026, https://www.bentoml.com/blog/a-guide-to-open-source-embedding-models
2. jina-embeddings-v3 - Search Foundation Models, fecha de acceso: febrero 9, 2026, https://jina.ai/models/jina-embeddings-v3/
3. README.md · Alibaba-NLP/gte-multilingual-base at main - Hugging Face, fecha de acceso: febrero 9, 2026, https://huggingface.co/Alibaba-NLP/gte-multilingual-base/blob/main/README.md
4. Jina Embeddings | LlamaIndex Python Documentation, fecha de acceso: febrero 9, 2026, https://developers.llamaindex.ai/python/framework/integrations/embeddings/jinaai_embeddings/
5. Support for jinaai/jina-embeddings-v3 embedding model · Issue #6922 - GitHub, fecha de acceso: febrero 9, 2026, https://github.com/ollama/ollama/issues/6922
6. BAAI/bge-m3 · Hugging Face, fecha de acceso: febrero 9, 2026, https://huggingface.co/BAAI/bge-m3
7. Bug: Error while converting BERT to GGUF: Can not map tensor 'bert.embeddings.LayerNorm.beta' · Issue #7924 · ggml-org/llama.cpp - GitHub, fecha de acceso: febrero 9, 2026, https://github.com/ggerganov/llama.cpp/issues/7924
8. [Bug] OllamaEmbedder not working with specific dimensions · Issue #5154 · agno-agi/agno, fecha de acceso: febrero 9, 2026, https://github.com/agno-agi/agno/issues/5154
9. Top embedding models on the MTEB leaderboard - Modal, fecha de acceso: febrero 9, 2026, https://modal.com/blog/mteb-leaderboard-article
10. Top Embedding Models in 2025 — The Complete Guide - Artsmart.ai, fecha de acceso: febrero 9, 2026, https://artsmart.ai/blog/top-embedding-models-in-2025/
11. MIRACL | Multilingual Information Retrieval Across a Continuum of Languages, fecha de acceso: febrero 9, 2026, https://project-miracl.github.io/
12. Ollama embedding model bge-m3 produces Nan output in some seemingly unrelated cases · Issue #13572 - GitHub, fecha de acceso: febrero 9, 2026, https://github.com/ollama/ollama/issues/13572
13. Pretrained multilingual model for sentence embedding with a Max Sequence Length > 128 · Issue #1476 - GitHub, fecha de acceso: febrero 9, 2026, https://github.com/UKPLab/sentence-transformers/issues/1476
14. AWS Marketplace: Nomic Embed Text v1.5 - Amazon.com, fecha de acceso: febrero 9, 2026, https://aws.amazon.com/marketplace/pp/prodview-xume634dhbnyu
15. BGE M3-Embedding: Multi-Lingual, Multi-Functionality, Multi-Granularity Text Embeddings Through Self-Knowledge Distillation - arXiv, fecha de acceso:

febrero 9, 2026, https://arxiv.org/html/2402.03216v3
16. Top 5 Local LLM Tools and Models in 2026 - Pinggy, fecha de acceso: febrero 9, 2026, https://pinggy.io/blog/top_5_local_llm_tools_and_models/
17. qwen3-embedding:0.6b - Ollama, fecha de acceso: febrero 9, 2026, https://ollama.com/library/qwen3-embedding:0.6b
18. KaLM-Embedding-V2: Superior Training Techniques and Data Inspire A Versatile Embedding Model - arXiv, fecha de acceso: febrero 9, 2026, https://arxiv.org/html/2506.20923v5
19. dengcao/Qwen3-Embedding-0.6B - Ollama, fecha de acceso: febrero 9, 2026, https://ollama.com/dengcao/Qwen3-Embedding-0.6B
20. Alibaba-NLP/gte-multilingual-base - Hugging Face, fecha de acceso: febrero 9, 2026, https://huggingface.co/Alibaba-NLP/gte-multilingual-base
21. mGTE: Generalized Long-Context Text Representation and Reranking Models for Multilingual Text Retrieval - arXiv, fecha de acceso: febrero 9, 2026, https://arxiv.org/html/2407.19669v1
22. milkey/gte - Ollama, fecha de acceso: febrero 9, 2026, https://ollama.com/milkey/gte
23. jina-embeddings-v3: Multilingual Embeddings With Task LoRA - arXiv, fecha de acceso: febrero 9, 2026, https://arxiv.org/html/2409.10173v3